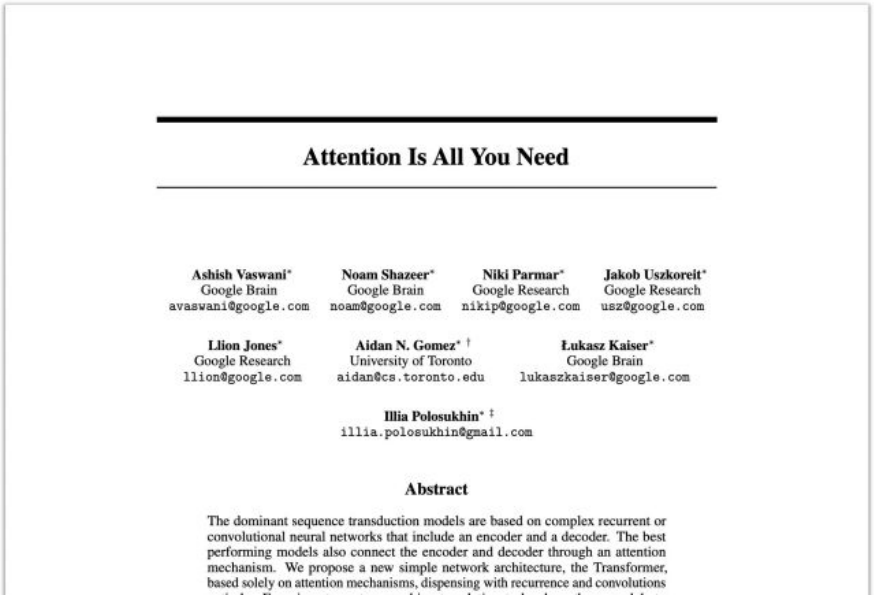
Transformers

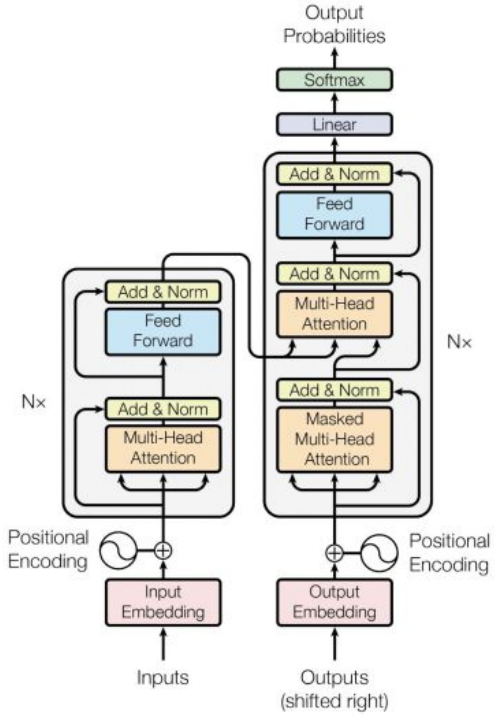


Attention Is All You Need

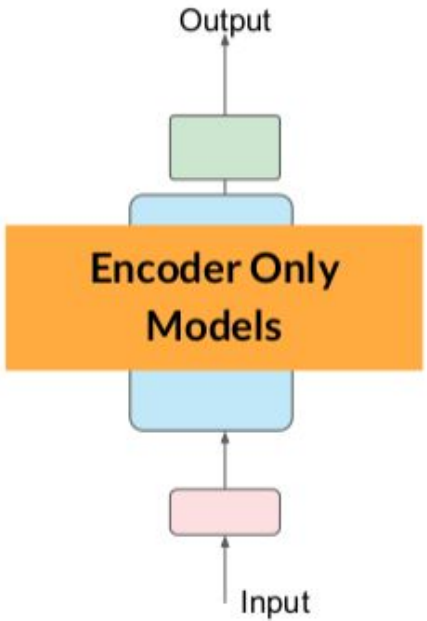
Ashish Vaswani* Google Brain avaswani@google.com
Noam Shazeer* Google Brain noam@google.com
Niki Parmar* Google Research nikip@google.com
Jakob Uszkoreit* Google Research usz@google.com
Llion Jones* Google Research llion@google.com
Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu
Łukasz Kaiser* Google Brain lukaszkaiser@google.com
Illia Polosukhin* ‡ illia.polosukhin@gmail.com

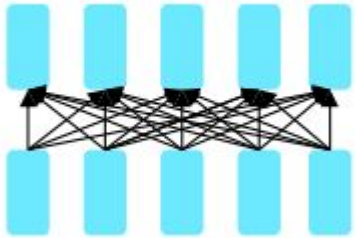
Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions



Types of Transformers Models



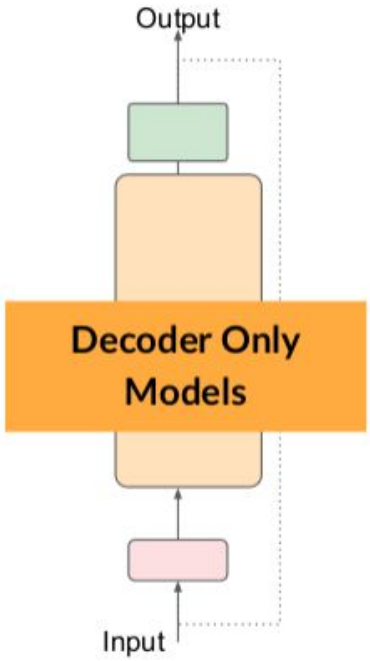


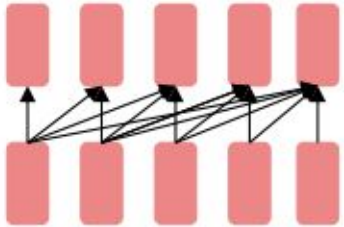
BERT family,
HuBERT

Good use cases:

- Sentiment analysis
- Named entity recognition
- Word classification

Types of Transformers Models



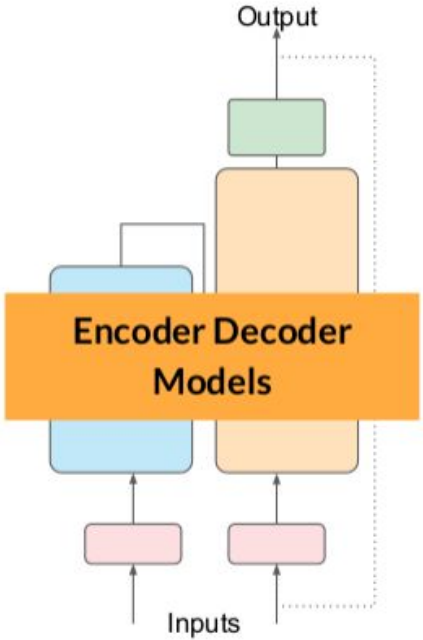


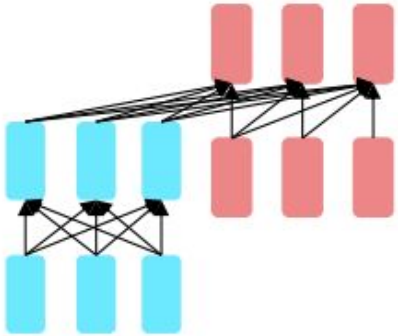
GPT, Claude,
Llama 2
Mixtral

Good use cases:

- Text generation
- Other emergent behavior
  - Depends on model size

Types of Transformers Models





Encoder-decoders

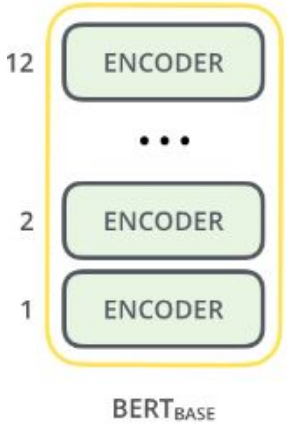
Flan-T5, Whisper

Good use cases:

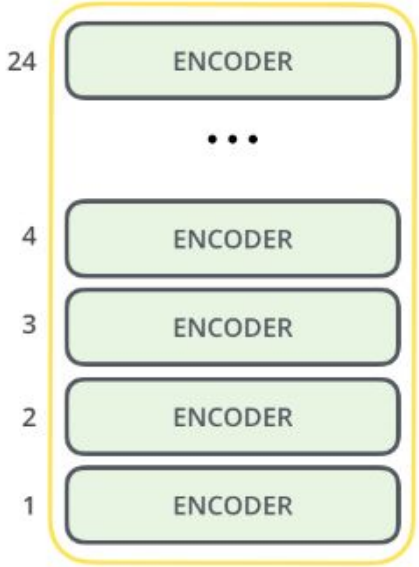
- Translation
- Text summarization
- Question answering

Encoder Only Model - BERT

Also Called as Encoders Only Model



$BERT_{BASE}$



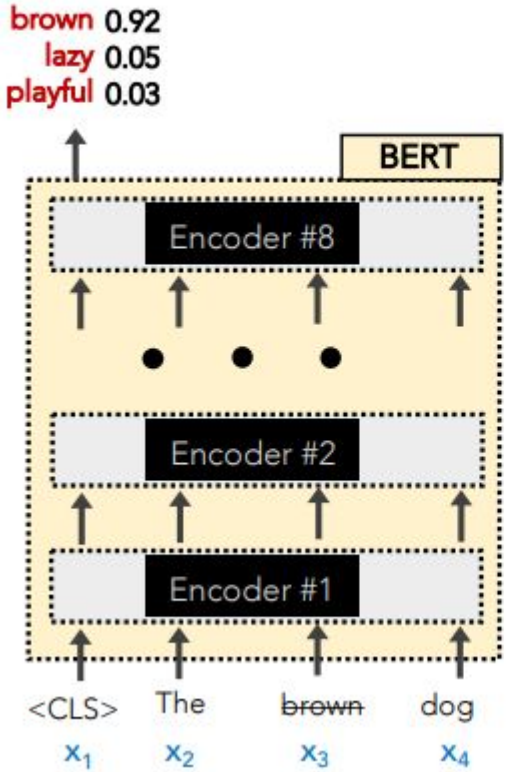
$BERT_{LARGE}$

To recap: all of this looks fancy, but ultimately it's just producing a very good **contextualized embedding** r_i of each word x_i

Why stop with just 1 Transformer Encoder? We could stack several!

How BERT Pretrained ? - MLM Approach

BERT



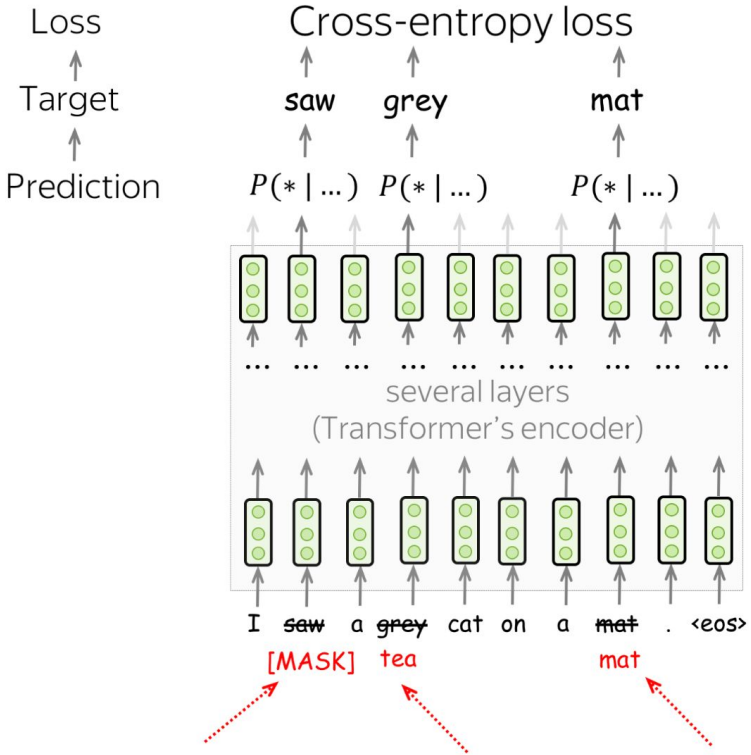
BERT has 2 training objectives:

1. Predict the **Masked word** (a la CBOW)

15% of all input words are randomly masked.

- 80% become [MASK]
- 10% become revert back
- 10% become are deliberately corrupted as wrong words

How BERT Pretrained ? - MLM Approach

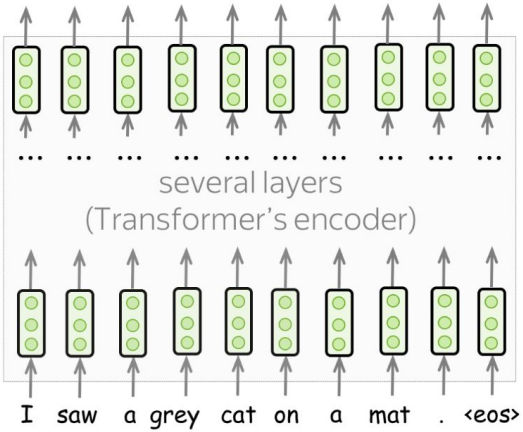


At each training step:

- pick randomly 15% of tokens
- replace each of the chosen tokens with something
- predict original chosen tokens

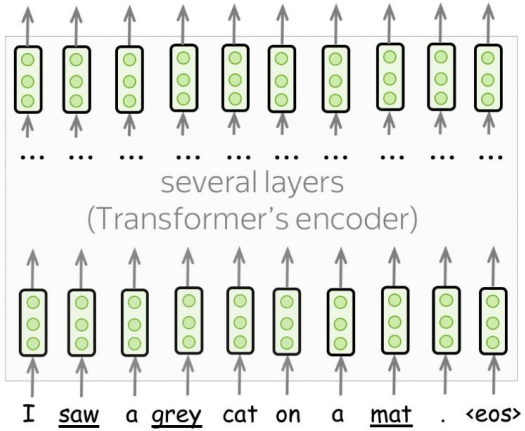
- **[MASK]**, with p = 80%
- Random token, with p = 10%
- Original token, with p = 10%

How BERT Pretrained ? - MLM Approach



At each training step:

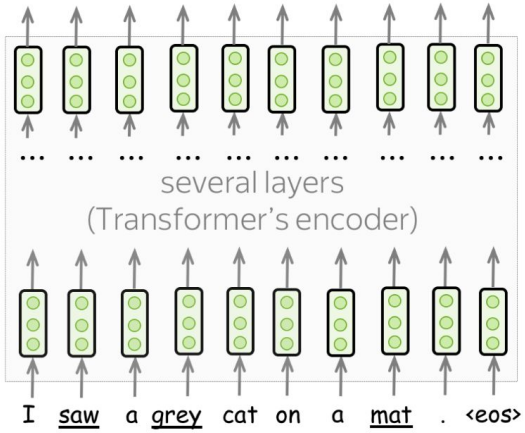
How BERT Pretrained ? - MLM Approach



At each training step:

- pick randomly 15% of tokens

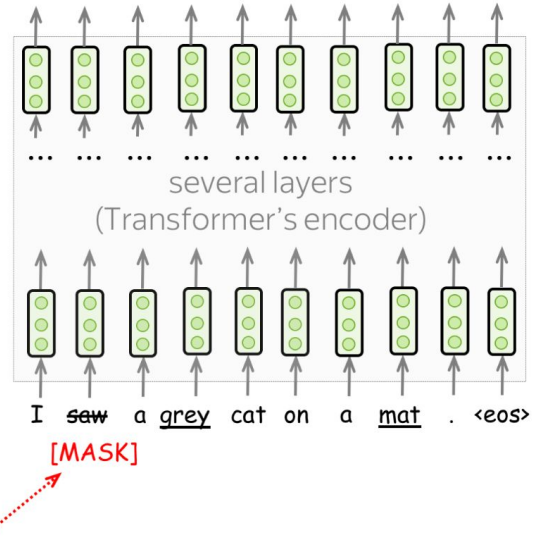
How BERT Pretrained ? - MLM Approach



At each training step:

- pick randomly 15% of tokens
- replace each of the chosen tokens with something

How BERT Pretrained ? - MLM Approach

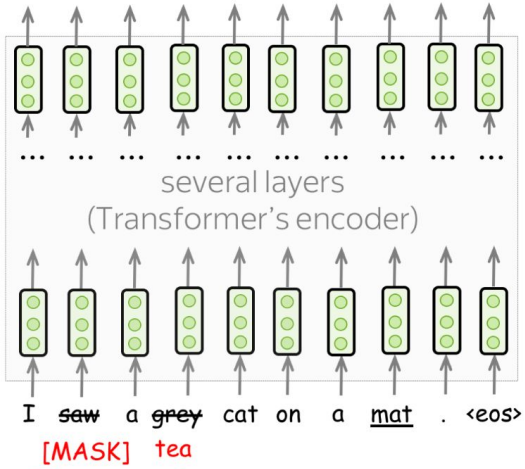


At each training step:

- pick randomly 15% of tokens
- replace each of the chosen tokens with something

- [MASK], with p = 80%

How BERT Pretrained ? - MLM Approach

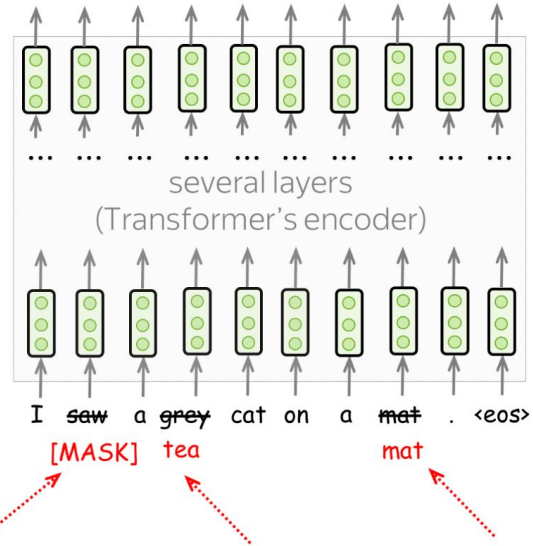


At each training step:

- pick randomly 15% of tokens
- replace each of the chosen tokens with something

- [MASK], with p = 80%
- Random token, with p = 10%

How BERT Pretrained ? - MLM Approach



At each training step:

- pick randomly 15% of tokens
- replace each of the chosen tokens with something

- **[MASK]**, with p = 80%
- Random token, with p = 10%
- Original token, with p = 10%

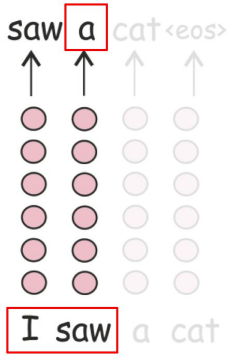
How BERT Pretrained ? - MLM Approach

Standard left-to-right language models predict the next token based only on previous tokens, so their final representations encode **only past context** and do not see future tokens.

In contrast, masked language models (MLMs) like **BERT** process the whole text at once with some tokens masked, making them **bidirectional** by utilizing both past and future context. Unlike **ELMo**, which trains two separate unidirectional models (left-to-right and right-to-left) and concatenates their representations to capture both contexts, BERT achieves bidirectionality with **a single model**, eliminating the need to train separate models.

Language Modeling

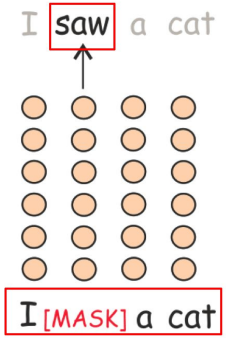
- Target: next token
- Prediction: $P(* \mid \texttt{I saw})$



left-to-right, does not see future

Masked Language Modeling

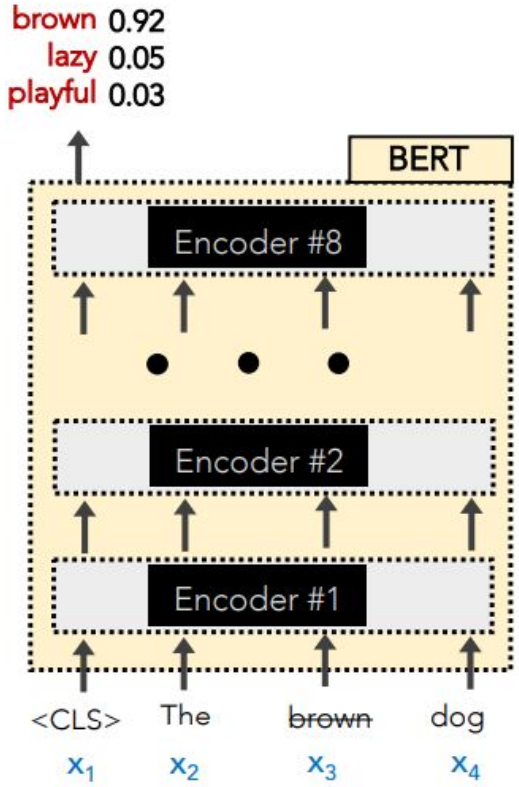
- Target: current token (the true one)
- Prediction: $P(* \mid \texttt{I [MASK] a cat})$



sees the whole text, but something is corrupted

How BERT Pretrained ? - NSP Approach

BERT



BERT has 2 training objectives:

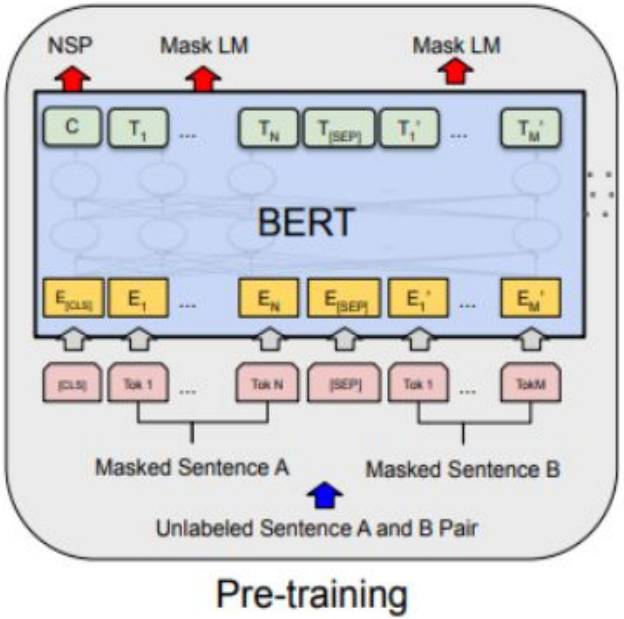
2. Two sentences are fed in at a time. Predict the if the second sentence of input truly follows the first one or not.

To learn *relationships* between sentences, predict whether Sentence B is actual sentence that proceeds Sentence A, or a random sentence

Sentence A = The man went to the store.
Sentence B = He bought a gallon of milk.
Label = IsNextSentence

Sentence A = The man went to the store.
Sentence B = Penguins are flightless.
Label = NotNextSentence

How BERT Pretrained ? - NSP Approach



Pre-training

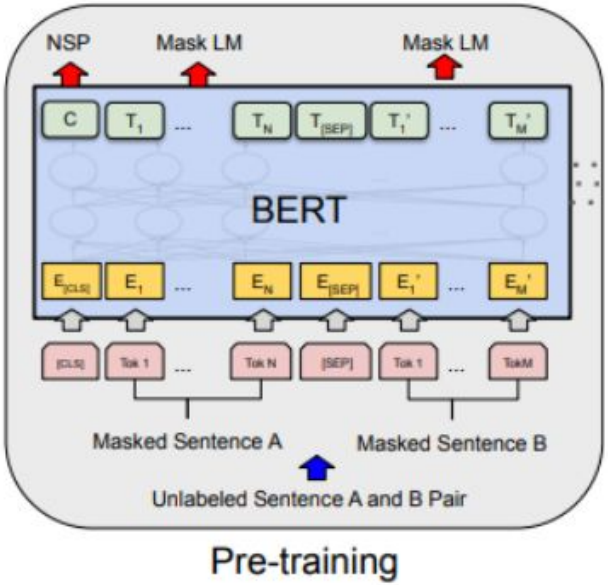
Every two sentences are separated by a <SEP> token.

50% of the time, the 2nd sentence is a randomly selected sentence from the corpus.

50% of the time, it truly follows the first sentence in the corpus.

How BERT Pretrained ? - NSP Approach - **WordPiece**

BERT



Pre-training

NOTE: BERT also embeds the inputs by their **WordPiece** embeddings.

WordPiece is a **sub-word tokenization** learns to merge and use characters based on which pairs maximize the likelihood of the training data if added to the vocab.

How BERT Pretrained ? - NSP Approach - **WordPiece**

What problem does Subword Tokenisation solve?

Tokenization cannot handle unseen words or rare words well. However, it may be too fine-grained missing some important information.

Subword Tokenisation is one of the solutions to overcome out-of-vocabulary (OOV). The subword is in between word and character. It is not too fine-grained while being able to handle unseen words and rare words.

Advantages of Subword Tokenization:

- Improved handling of out-of-vocabulary words.
- Better representation of complex linguistic structures.
- Enhanced model generalization across different languages and domains.

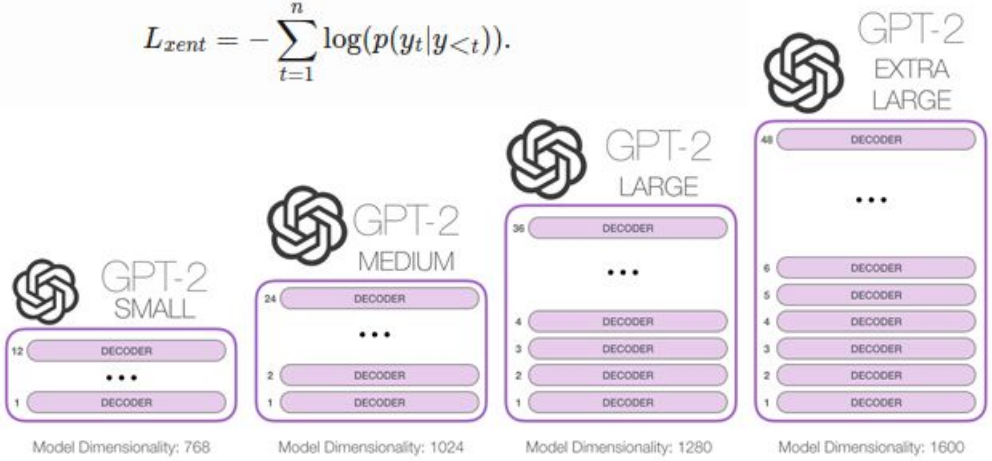
Aspect	Tokenization	Subword Tokenization
Definition	The process of splitting text into individual words, phrases, or meaningful units.	A method that breaks words into smaller, meaningful units (subwords), often to handle unknown words or to provide a more fine-grained representation of the text.
Purpose	To simplify text analysis by reducing it to its basic units or tokens.	To capture morphological nuances and handle rare or out-of-vocabulary words by breaking them down into smaller subwords.
Method	Splits text based on spaces and punctuation, treating each word or symbol as a separate token.	Uses algorithms to split words into frequently occurring subwords or character sequences, ensuring that even unseen words can be represented.
Examples	- "I love NLP" -> ["I", "love", "NLP"] - "Happy coding!" -> ["Happy", "coding", "!"]	- "unhappiness" -> ["un", "happi", "ness"] - "transformer" -> ["trans", "form", "er"]
Applications	Basic text processing, word-based indexing, or text analysis.	Machine translation, text generation, and language modeling, especially in languages with rich morphology or when dealing with technical jargon.
Advantages	Simple and straightforward, works well for languages with clear word boundaries.	More robust to morphological variations, can handle unseen or rare words, improves model generalization.
Disadvantages	Struggles with out-of-vocabulary words, may not capture morphological variations effectively.	Can result in a higher number of tokens for processing, potentially leading to longer computational times and complexity.

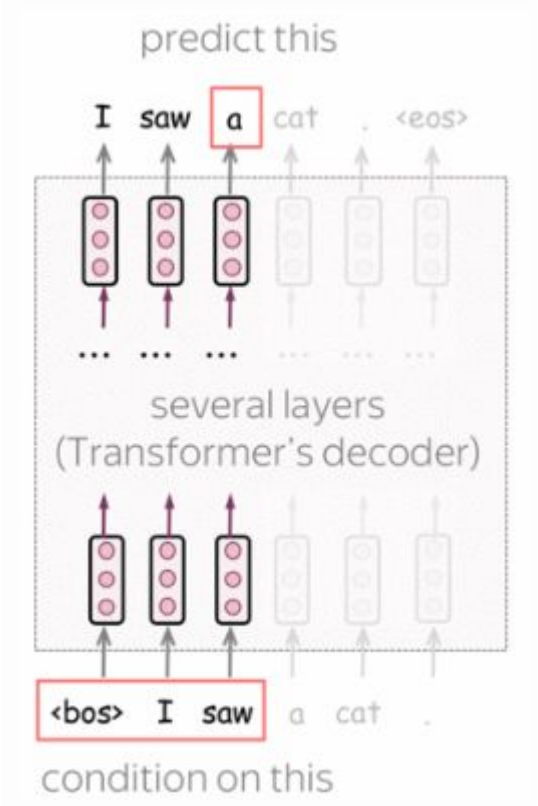
Decoder Only Model - GPT

GPT is a Transformer-based left-to-right language model. The architecture is a 12-layer Transformer decoder (without decoder-encoder attention).

Formally, if $y_1, \ldots, y_n$ is a training token sequence, then at the timestep t a model predicts a probability distribution $p^{(t)} = p(*|y_1, \ldots, y_{t-1})$. The model is trained with the standard cross-entropy loss, and the loss for the whole sequence is

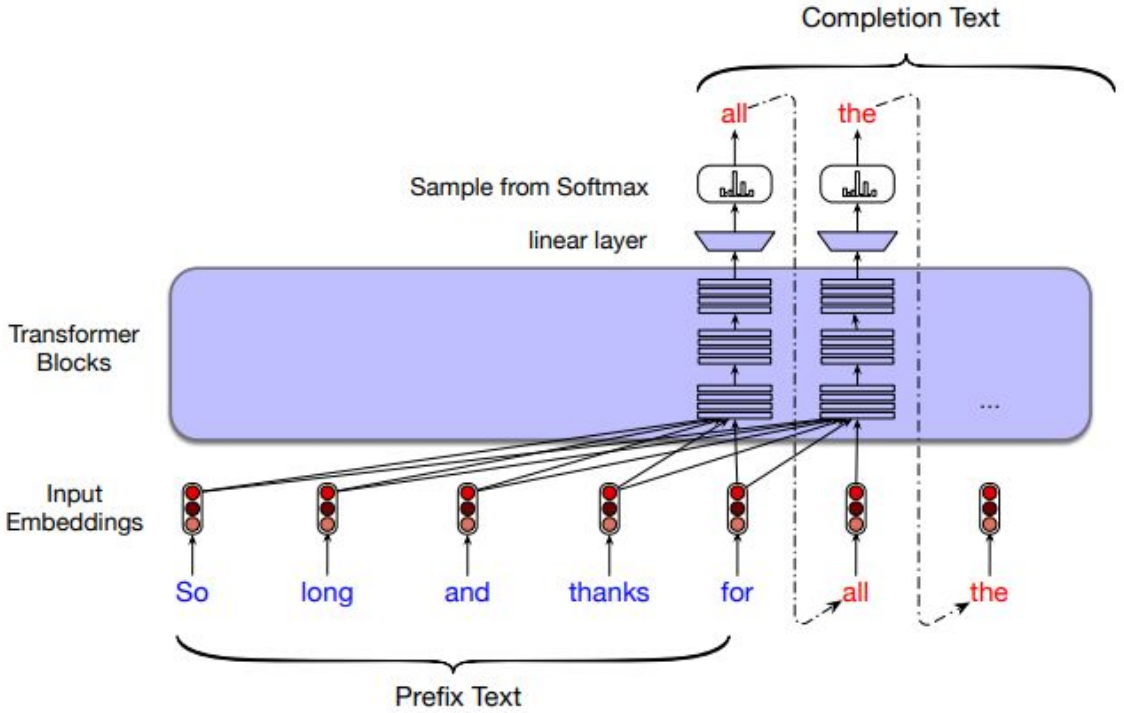
$$L_{xent} = -\sum_{t=1}^{n} \log(p(y_t|y_{<t})).$$





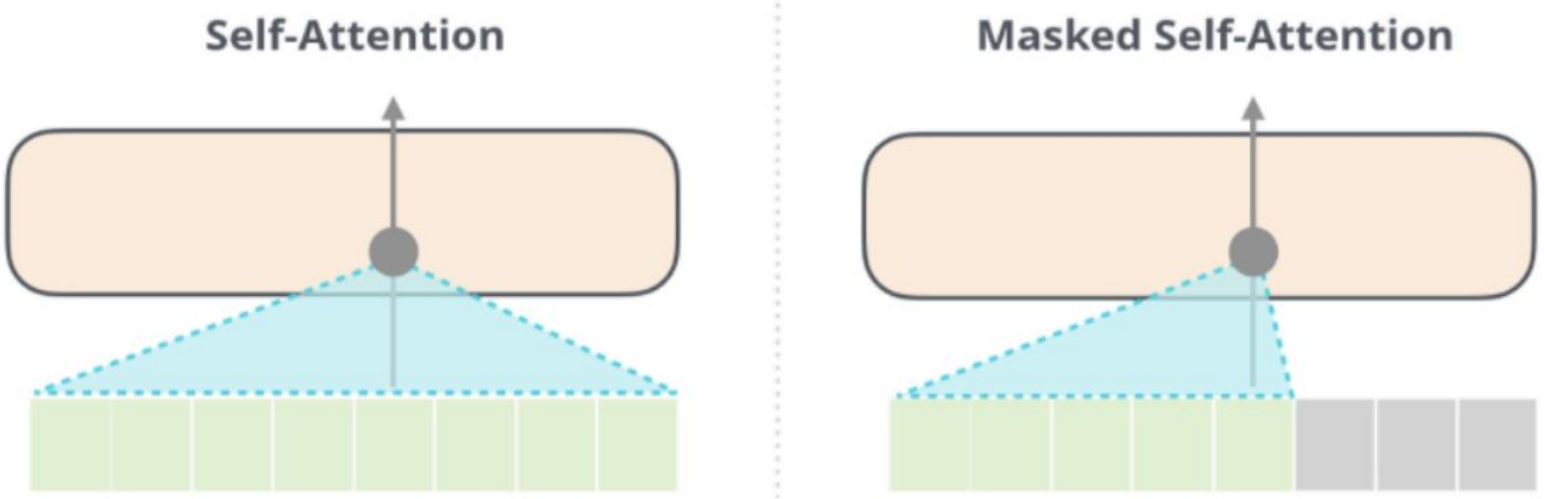
Decoder Only Model - GPT

Conditional Generation: Generating text conditioned on previous text!



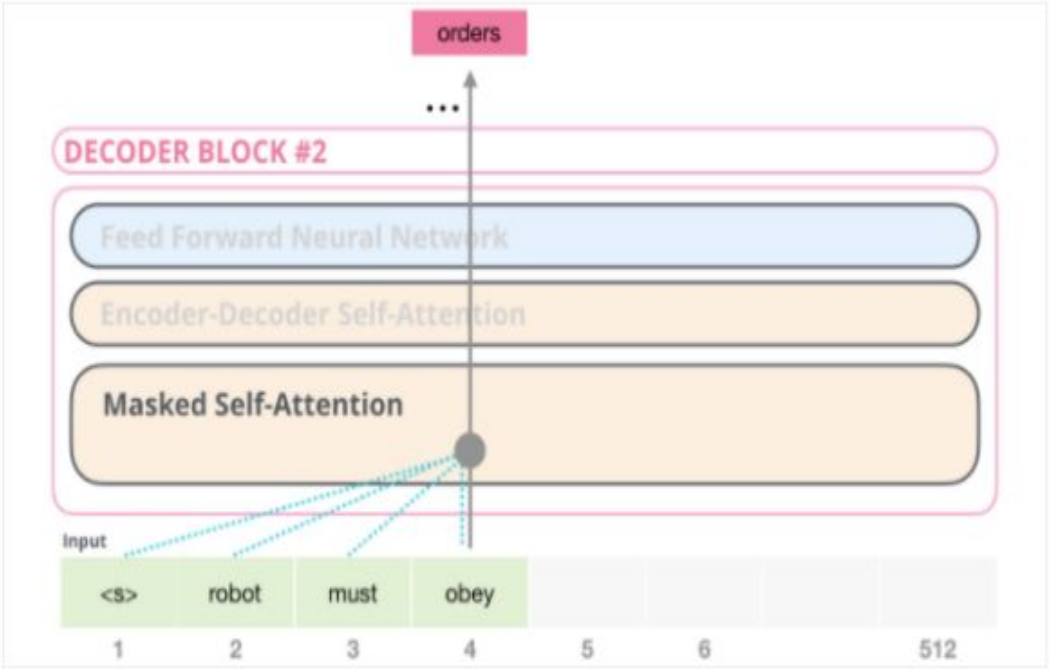
Decoder Only Model - GPT - Masked Attention

As it processes each word/token, it cleverly **masks** the "future" words and conditions itself on the previous words



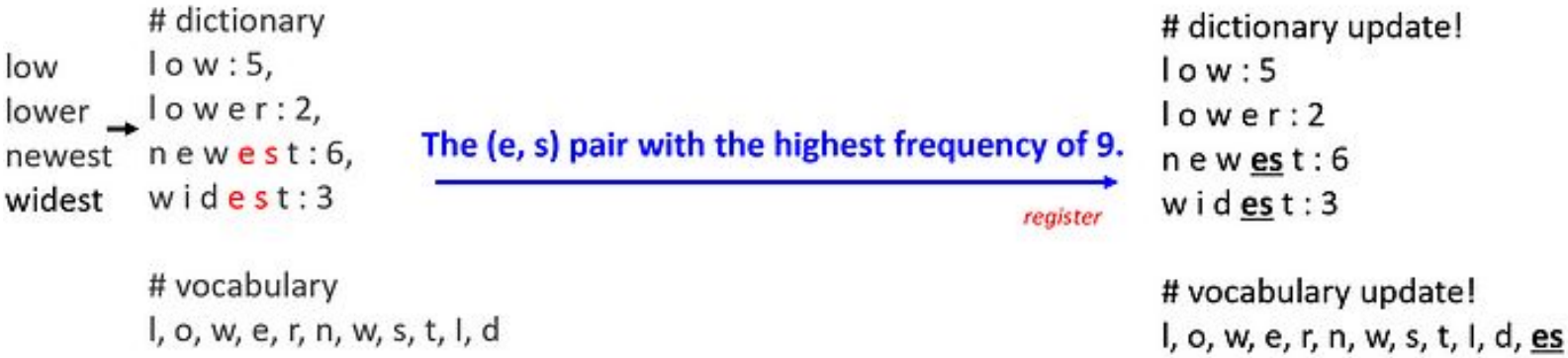
How GPT pretrained?

As it processes each word/token, it cleverly **masks** the "future" words and conditions itself on the previous words



How GPT pretrained? - Byte Pair Encoding

- Technically, it doesn't use words as input but Byte Pair Encodings (sub-words), similar to BERT's WordPieces.
- Includes positional embeddings as part of the input, too.
- Easy to fine-tune on your own dataset (language)



Fine Tuning/Transfer Learning in NLP in General

GloVe, Word2Vec → CoVe, ELMo → GPT, BERT

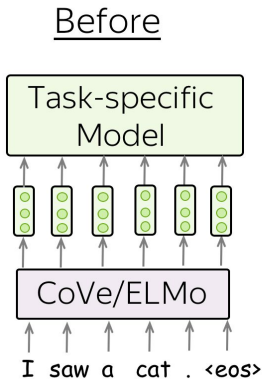
Great idea 1

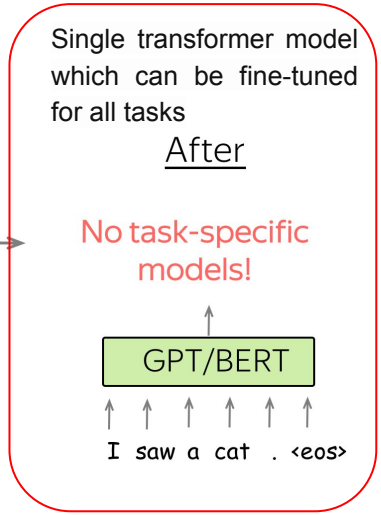
What is encoded	words → words in context	words in context
How it is used for downstream tasks	Input for task-specific models	Input for task-specific models → Instead of task-specific models

Great idea 2

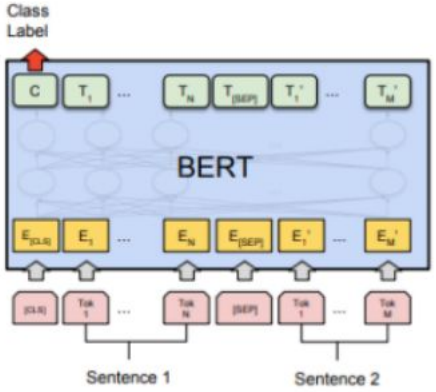
The two great ideas:

- **what is encoded: from words to words-in-context (the transition from Word2Vec/GloVe/etc. to Cove/ELMo);**
- **usage for downstream tasks: from replacing only word embeddings in task-specific models to replacing entire task-specific models (the transition from Cove/ELMo to GPT/BERT).**

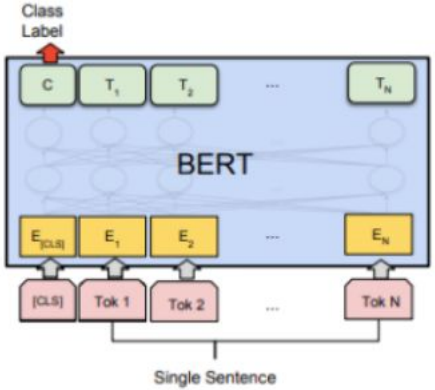




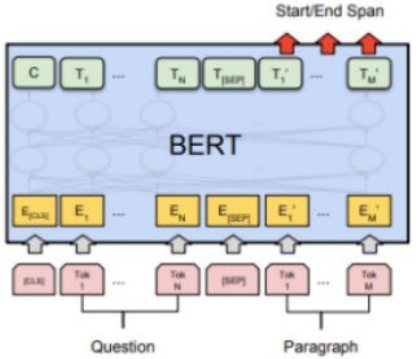
Fine tuning transformer models - BERT for downstream tasks



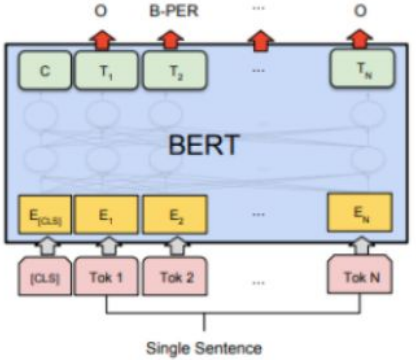
(a) Sentence Pair Classification Tasks: MNLI, QQP, QNLI, STS-B, MRPC, RTE, SWAG



(b) Single Sentence Classification Tasks: SST-2, CoLA



(c) Question Answering Tasks: SQuAD v1.1



(d) Single Sentence Tagging Tasks: CoNLL-2003 NER

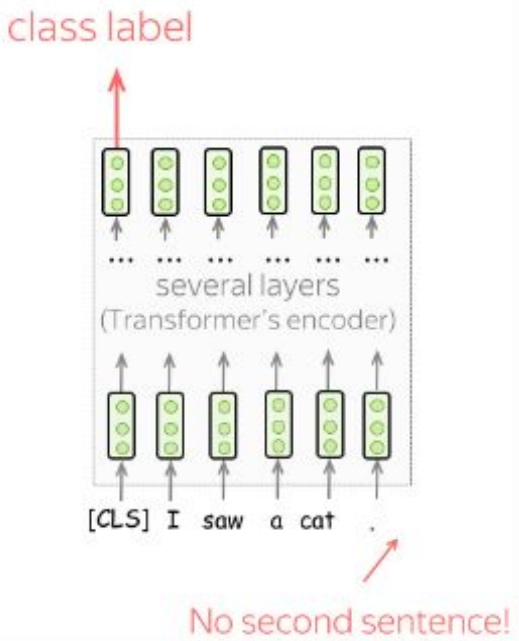
Fine tuning transformer models - BERT for downstream tasks

Single sentence classification

To classify individual sentences, feed the data as shown on the illustration and predict the label from the final representation of the [CLS] token.

Examples of tasks:

- SST-2 - binary sentiment classification (the one we saw in the Text Classification lecture);
- CoLA (Corpus of Linguistic Acceptability) - say whether a sentence is linguistically acceptable.



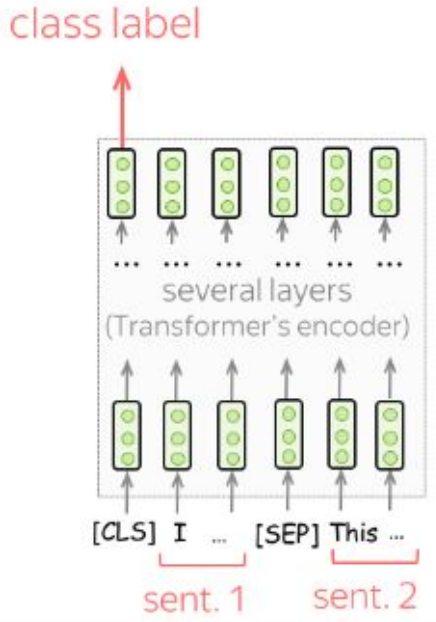
Fine tuning transformer models - BERT for downstream tasks

Sentence Pair Classification

To classify pairs of sentences, feed the data as you did in training. Similar to the single sentence classification, predict the label from the final representation of the **[CLS]** token.

Examples of tasks:

- SNLI - entailment classification. Given a pair of sentences, say if the second is an **entailment**, **contradiction**, or **neutral**);
- QQP (Quora Question Pairs) - given two questions, say if they are semantically equivalent;
- STS-B - given two sentences, return a similarity score from 1 to 5.

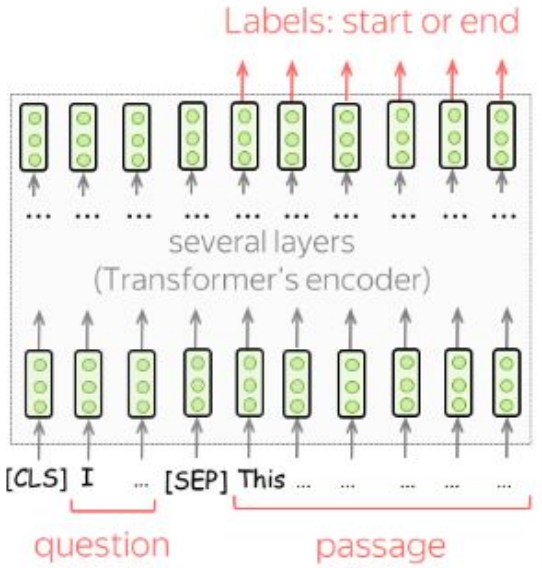


Fine tuning transformer models - BERT for downstream tasks

Question Answering

For QA, the BERT authors used only one dataset, SQuAD v1.1. In this task, you are given a text passage and a question. The answer to this question is always a part of the passage, and the task is to find the correct segment of the passage.

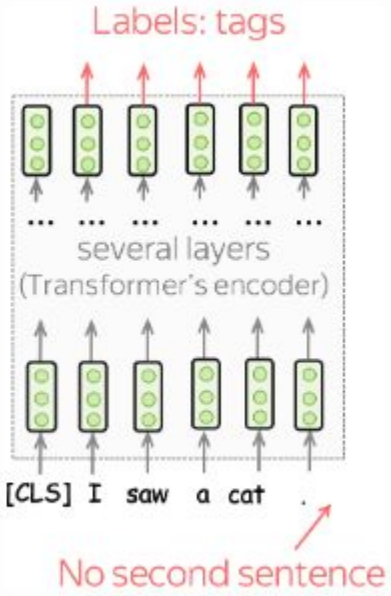
To use BERT for this task, feed a question and a passage as shown in the illustration. Then, for each token in the passage use final BERT representations to predict whether this token is the start or the end of the correct segment.



Fine tuning transformer models - BERT for downstream tasks

Single sentence tagging

In tagging tasks, you have to predict tags for each token. For example, in Named Entity Recognition (NER), you have to predict if a word is a named entity and its type (e.g., **location**, **person**, etc).



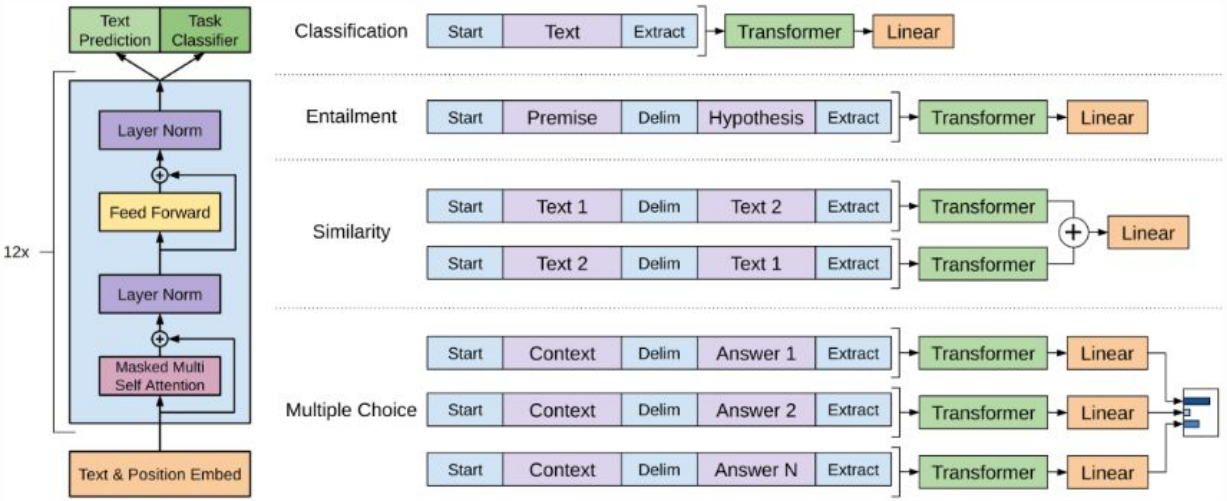
Fine tuning transformer models - GPT for downstream tasks

Fine-Tuning: Using GPT for Downstream Tasks

The fine-tuning loss consists of the task-specific loss, as well as the language modeling loss:

$$L = L_{xent} + \lambda \cdot L_{task}.$$

In the fine-tuning stage, the model architecture stays the same except for the final linear layer. What changes is the input format for each task: look at the illustration below.



Single sentence classification

To classify individual sentences, just feed the data as in training and predict the label from the final representation of the last input token.

Examples of tasks:

- SST-2 - binary sentiment classification (the one we saw in the Text Classification lecture);
- CoLA (Corpus of Linguistic Acceptability) - say whether a sentence is linguistically acceptable.

Sentence pairs classification

To classify sentence pairs, feed the two fragments with a special token-separator (e.g. **delim**). Then, predict the label from the final representation of the last input token.

Examples of tasks:

- SNLI - entailment classification. Given a pair of sentences, say if the second is an **entailment**, **contradiction** or **neutral**);
- QQP (Quora Question Pairs) - given two questions say if they are semantically equivalent;
- STS-B - given two sentences return a similarity score from 1 to 5.

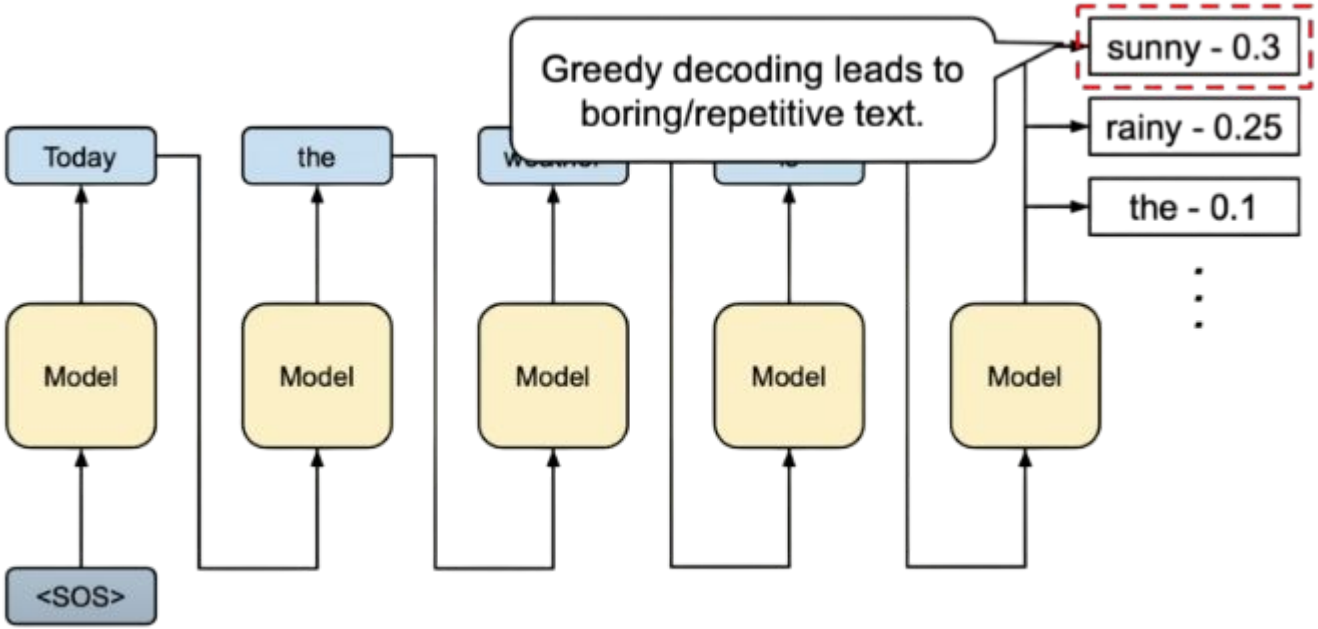
Question answering and commonsense reasoning

In these tasks, we are given a context document z, a question q, and a set of possible answers $\{a_k\}$. Concatenate document and question, and after the **delim** token add a possible answer. For each of the possible answers, process the corresponding sequences independently with the GPT model; then normalize via a softmax layer to produce an output distribution over possible answers.

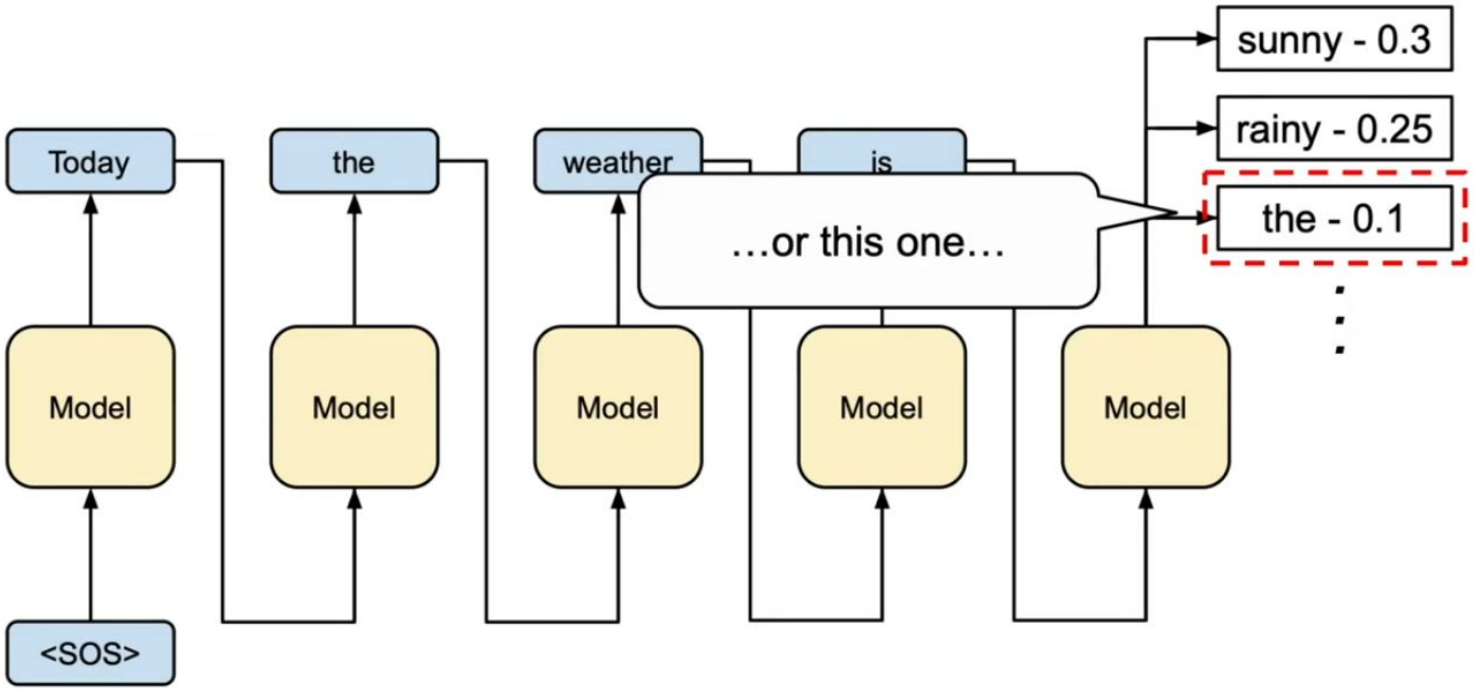
Examples of tasks:

- RACE - reading comprehension. Given a passage, a question and several answer options, pick the correct one.
- Story Cloze - story understanding and script learning. Given a four-sentence story and two possible endings, choose the correct ending to the story.

GPT decoding strategies : Greedy decoding



GPT decoding strategies



TEMPERATURE

Controls the randomness in the sampling process.

$$s(x_i) = \frac{e^{x_i}}{\sum_{i=1}^{N} e^{x_j}}$$

Softmax function

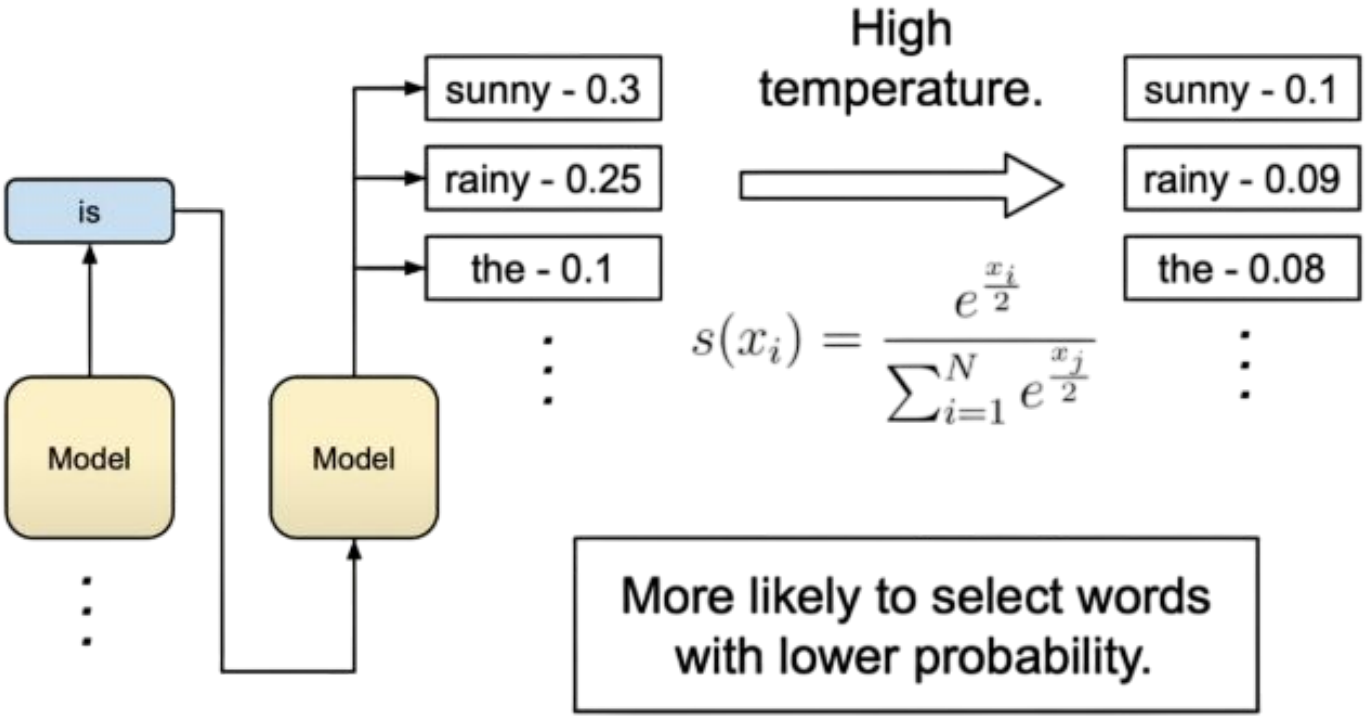
TEMPERATURE

Controls the randomness in the sampling process.

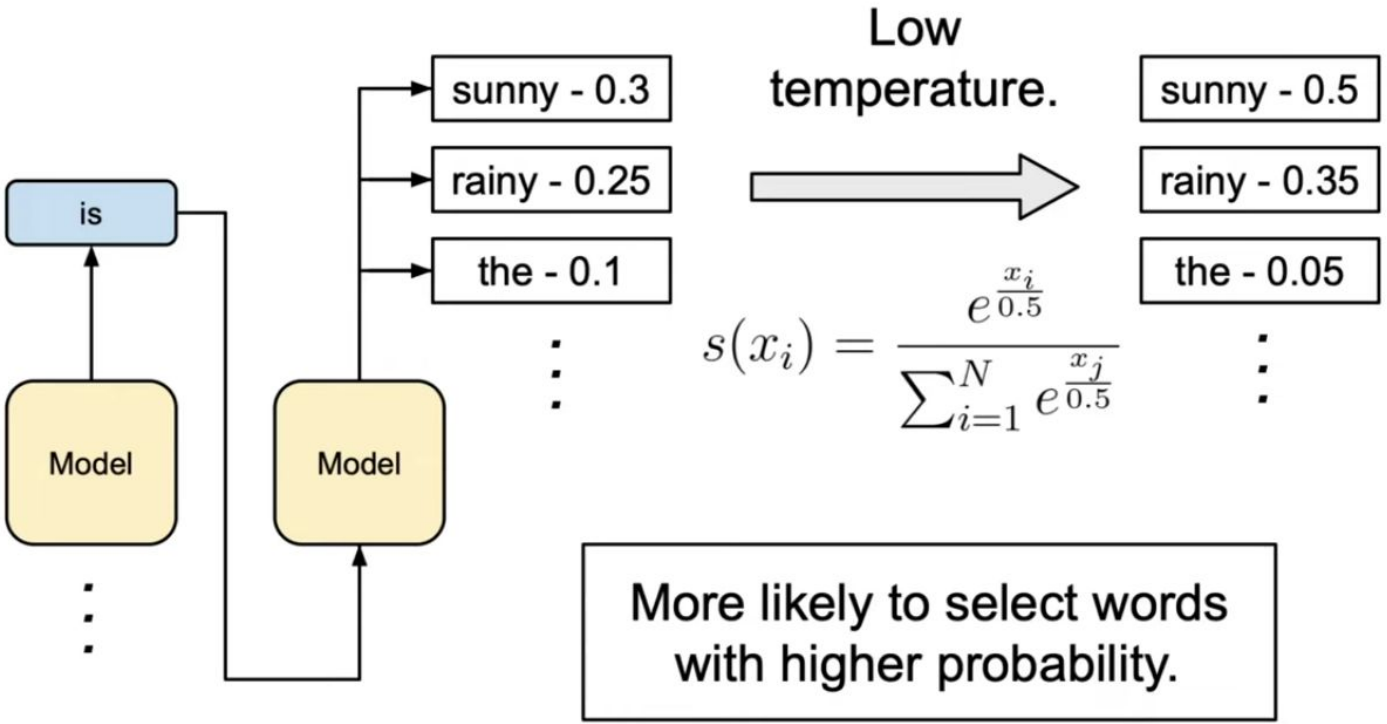
$$s(x_i) = \frac{e^{\frac{x_i}{\theta}}}{\sum_{i=1}^{N} e^{\frac{x_j}{\theta}}}$$

temperature

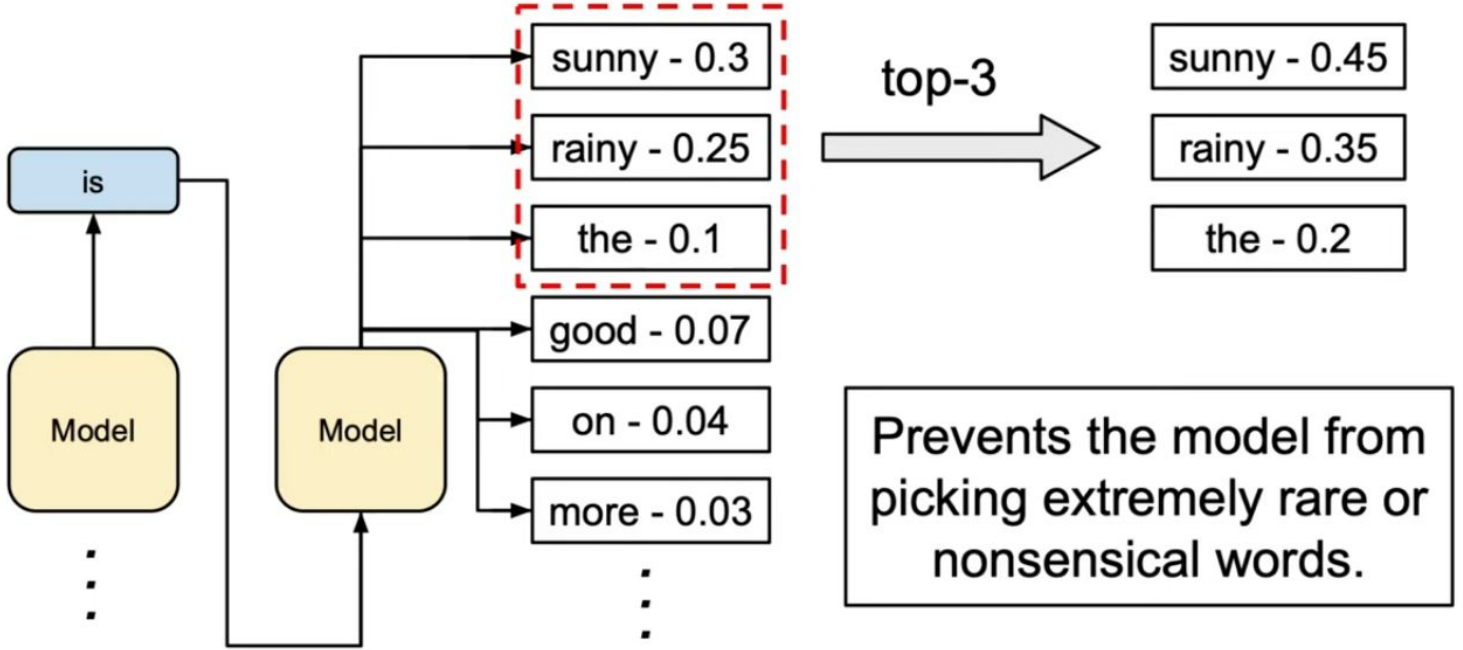
GPT decoding strategies



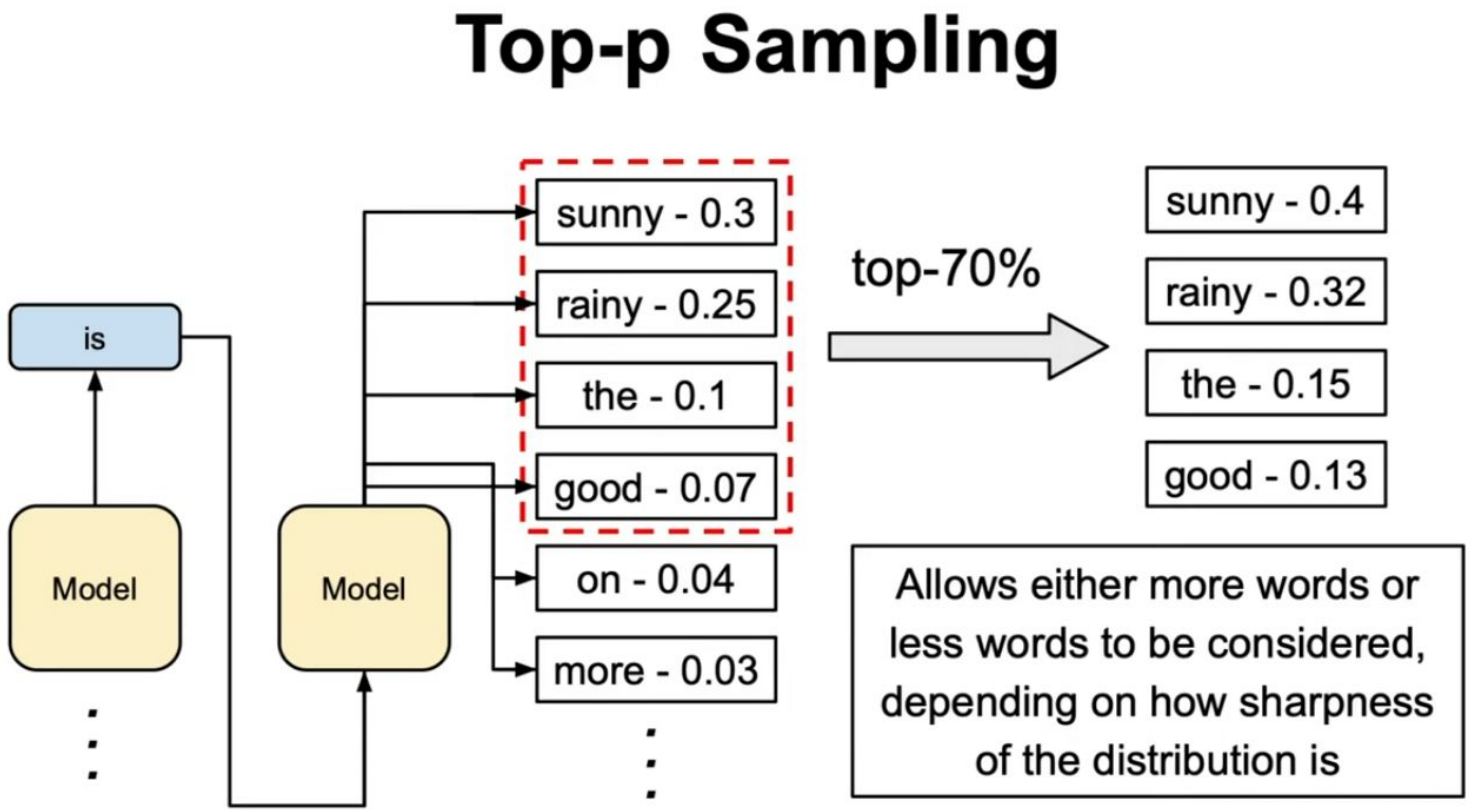
GPT decoding strategies



Top-k Sampling



GPT decoding strategies



GPT decoding strategies

Temperature	Top-k	Top-p
Pros: • Enhanced Creativity • Avoid rigidity	***Pros:*** • Controlled diversity • Reduced nonsense	***Pros:*** • Adaptive to probs • Ensured diversity
Cons: • Loss of coherence • Nonsensical text	***Cons:*** • Risk of repetitiveness • Sensitivity to k-value	***Cons:*** • Complex param tune • Coherence loss